

今年的考试试题安排

一、基本概念与理解 (40)

- 目标1：掌握算法基本概念、归纳法、分治法、动态规划、贪心算法、**NP**完全问题、回溯法、随机算法。 (30)
- 目标4：能够结合具体应用案例，合理选择、改进经典的算法，并能从算法角度分析和理解计算机相关专业工程实践对环境、社会可持续发展的影响。 (10)

今年的考试试题安排

二、算法分析与优化 (30)

- 目标2: 能够针对具体应用问题, 在多种可选方案中, 根据问题的约束条件, 分析各种方案在数据时间、空间复杂度及算法效率上的利弊, 选择合理的算法并进行优化处理。 (30)

今年的考试试题安排

三、算法设计与分析 (30)

- 目标3: 能够结合具体应用案例, 合理选择、改进经典的算法, 并能运用经典算法设计出复杂算法, 使之针对具体应用能够高效地存储和处理数据, 并对算法进行有效分析和评价。 (30)

《算法设计技巧与分析》

第6章 动态规划

朱友文

zhuyw@nuaa.edu.cn

6.1 引言

➤ 递归算法的不足之处

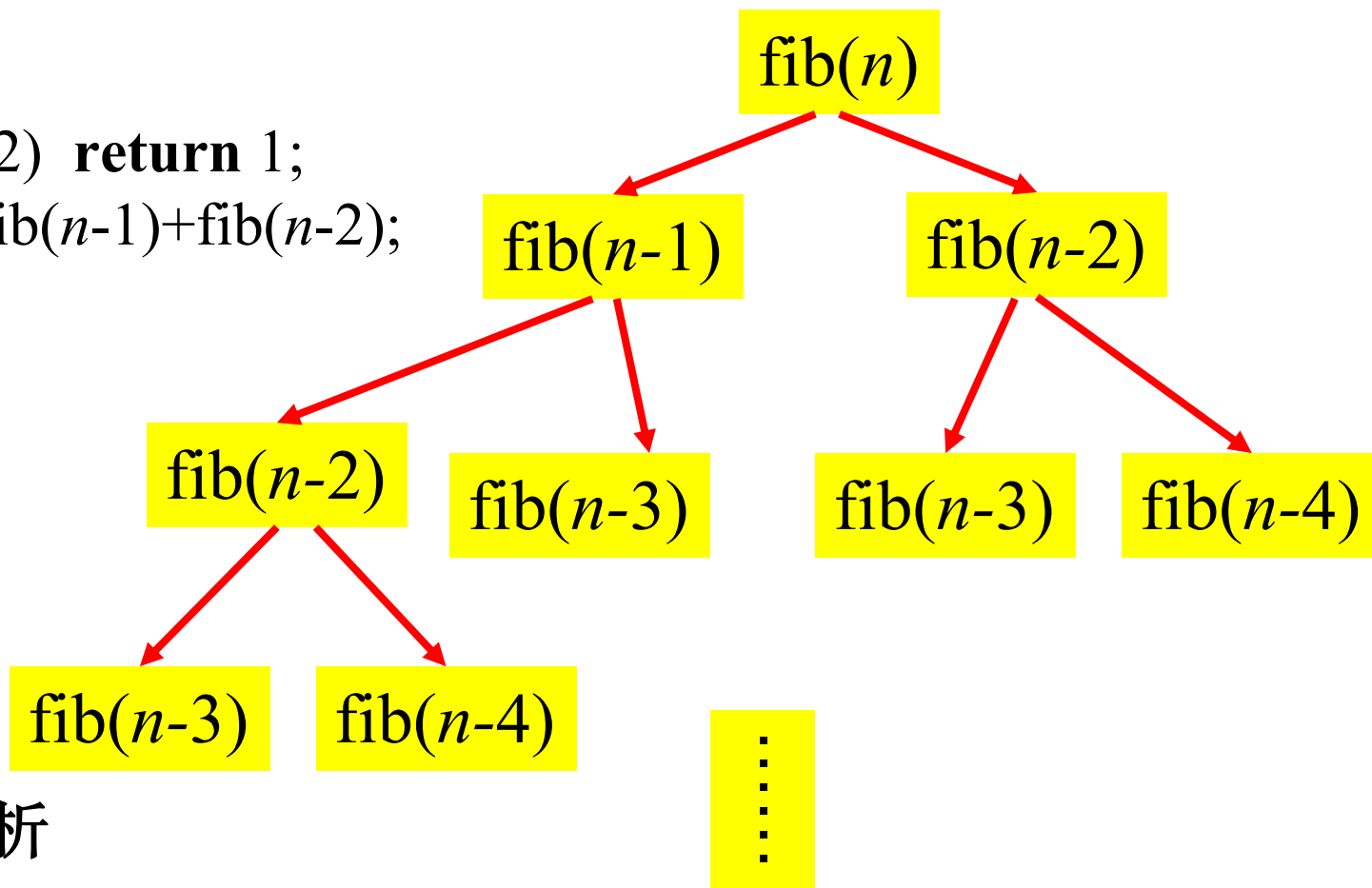
➤ Fibonacci数列: $f_1=1, f_2=1$, 求 $f(n)$

递归的方法

```
long fib( $n$ )  
{  
    if ( $n=1$  or  $n=2$ ) return 1;  
    else return fib( $n-1$ )+fib( $n-2$ );  
}
```

6.1 引言

```
long fib(n)
{
    if (n=1 or n=2) return 1;
    else return fib(n-1)+fib(n-2);
}
```



时间复杂度分析

$$f(n) = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n$$

6.1 引言

自底向上的计算方法

```
long fib(n)  
{ if (n=1 or n=2 ) return 1;  
  f1 = 1;   f2 =1;  
  for ( i = 3  to n )  
  {   result = f1+f2;  
      f1 = f2;  
      f2 = result;  
  }//end for  
  return result;  
}
```

时间复杂度: $O(n)$

6.1 引言

你还记得时间复杂度为 $O(\log n)$ 的 f_n 求值算法吗？

6.1 引言

➤ 二项式系数

$$\binom{n}{k} = \begin{cases} 1 & \text{若 } k=0 \text{ 或 } k=n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{若 } 0 < k < n \end{cases}$$

```
long bin(n, k)
{
    if (n=k or k=0) return 1;
    else return bin(n-1, k-1)+bin(n-1, k);
}
```

6.1 引言

二项式系数的自底向上计算方法

$$\binom{n}{k} = \begin{cases} 1 & \text{若 } k=0 \text{ 或 } k=n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{若 } 0 < k < n \end{cases}$$

n											
k		1	2	3	4	5	6	7	8	9	10
0		1	1	1	1	1	1	1	1	1	1
1		1									
2			1								
3				1							
4					1						
5						1					??

6.1 引言

➤ 二项式系数

```
long bin1( $n, k$ ) //  $A[0, \dots, k][0, \dots, n]$ 
{
     $A[0][0, \dots, n] = 1;$ 
    for ( $i=1; i \leq k; i++$ )  $A[i][i] = 1;$ 
    for ( $i=1; i \leq k; i++$ )
        for ( $j=i+1; j \leq n; j++$ )
             $A[i][j] = A[i-1][j-1] + A[i][j-1];$ 
    return  $A[k][n];$ 
}
```

时间复杂度: $(2n-1-k)k/2$

6.1 引言

二项式系数的自底向上计算方法

$$\binom{n}{k} = \begin{cases} 1 & \text{若 } k=0 \text{ 或 } k=n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{若 } 0 < k < n \end{cases}$$

n											
k		1	2	3	4	5	6	7	8	9	10
0		1	1	1	1	1	1	1	1	1	1
1		1									
2			1								
3				1							
4					1						
5						1					

6.1 引言

➤ 二项式系数

```
long bin2( $n, k$ ) //  $A[0, \dots, k][0, \dots, n]$ 
```

```
{  $A[0][0, \dots, n] = 1;$ 
```

```
  for ( $i=1; i \leq k; i++$ )  $A[i][i] = 1;$ 
```

```
  for ( $i=1; i \leq k; i++$ )
```

```
    for ( $j=i+1; j \leq i+n-k; j++$ )
```

```
       $A[i][j] = A[i-1][j-1] + A[i][j-1];$ 
```

```
  return  $A[k][n];$ 
```

```
}
```

时间复杂度: $(n-k)k$

6.1 引言

二项式系数的自底向上计算方法

$$\binom{n}{k} = \begin{cases} 1 & \text{若 } k=0 \text{ 或 } k=n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{若 } 0 < k < n \end{cases}$$

n											
k		1	2	3	4	5	6	7	8	9	10
0		1	1	1	1	1	1	1	1	1	1
1		1									
2			1								
3				1							
4					1						
5						1					

6.1 引言

➤ 二项式系数

```
long bin2-2(n, k)  // A[0, ..., k][0, ..., n]  
{  A[0][0, ..., n] = 1;  
    for (i=1; i ≤ k; i++)  A[i][i] = 1;  
    for (d=1; d ≤ n-k; d++)  
        for ( i=1; i ≤ k; i++) {  
            j = i + d;  
            A[i][j] = A[i-1][j-1] + A[i][j-1];  
        }  
    return A[k][n];  
}
```

时间复杂度: $(n-k)k$

6.1 引言

二项式系数的自底向上计算方法

$$\binom{n}{k} = \begin{cases} 1 & \text{若 } k=0 \text{ 或 } k=n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{若 } 0 < k < n \end{cases}$$

n		1	2	3	4	5	6	7	8	9	10
k	0	1	1	1	1	1	1	1	1	1	1
1	1										
2		1									
3			1								
4				1							
5					1						??

6.1 引言

➤ 二项式系数

```
long bin4(n, k)
{
    A[1, 2, ..., k] = 1; //使数组每个元素等于1
    for (d=1; d ≤ n-k; d++)
    {
        A[1] = A[1] + 1;
        for (j=2; j ≤ k; j++)
            A[j] = A[j] + A[j-1];
    }
    return A[k];
}
```

空间复杂度: $O(k)$

6.1 引言

二项式系数的自底向上计算方法

$$\binom{n}{k} = \begin{cases} 1 & \text{若 } k=0 \text{ 或 } k=n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{若 } 0 < k < n \end{cases}$$

n		1	2	3	4	5	6	7	8	9	10
k	0	1	1	1	1	1	1	1	1	1	1
1	1										
2		1									
3			1								
4				1							
5					1						??

6.1 引言

➤ 二项式系数

```
long bin3(n, k)
{
    A[1, ..., n-k] = 1;
    for (i=1; i ≤ k; i++)
    {
        A[1] = A[1] + 1;
        for (j=2; j ≤ n-k; j++)
            A[j] = A[j] + A[j-1];
    }
    return A[n-k];
}
```

空间复杂度: $O(n-k)$

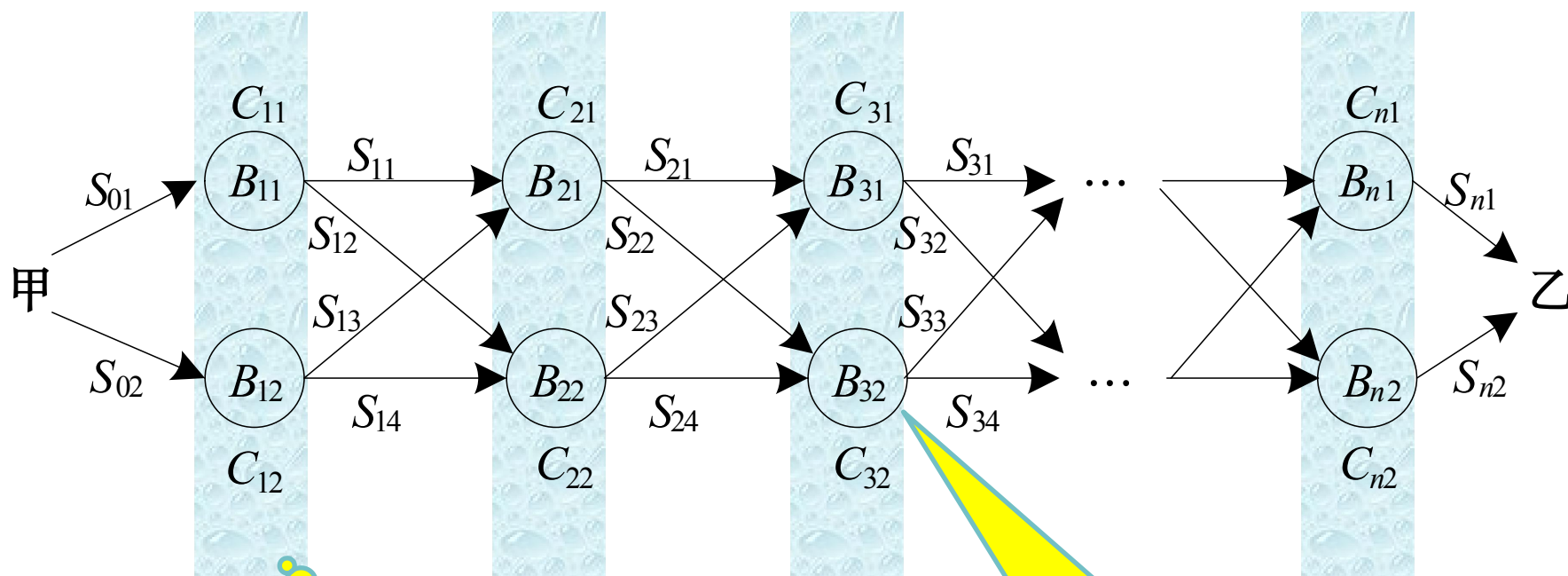
6.1 引言

启发

递归的思路分析问题，
根据依赖关系，自底向上的方法实现。

C_{ij} 和 S_{ij} 分别表示经过某个桥和路的代价

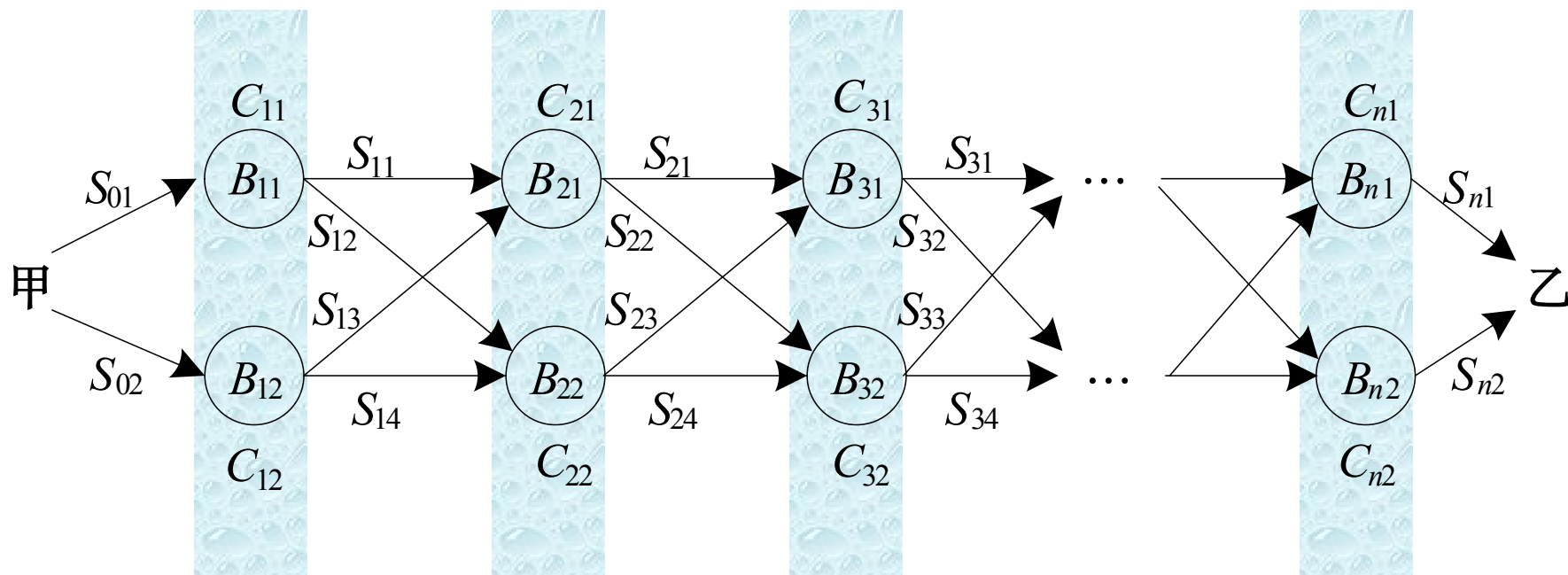
➤ 问题：哪条路代价最小？桥的代价=？



这表示一条
河

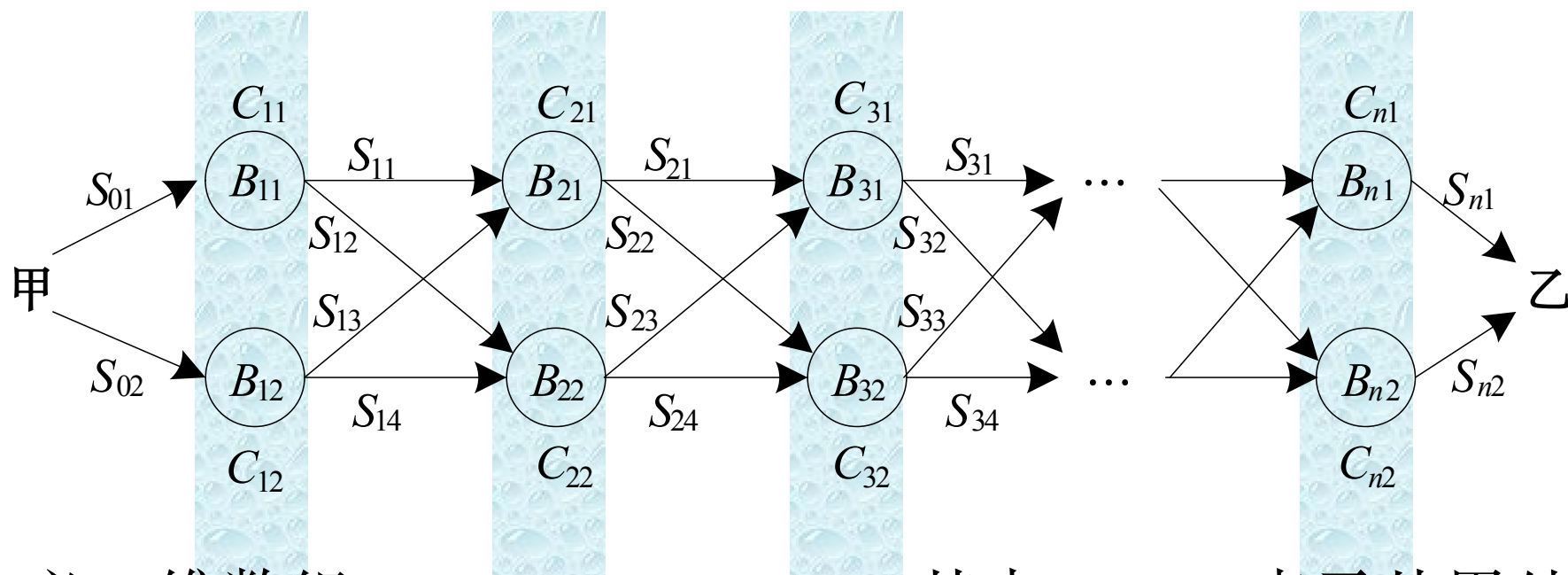
每个 B_{ij} 表示河上的
桥

➤ 问题: 哪种走法的代价最少? 最少代价=?



蛮力法? 指数量级

➤ 问题：哪种走法的代价最少？最少代价=？



定义二维数组 $M[1, 2, \dots, n][1, 2]$, 其中 $M[i][j]$ 表示从甲处到达桥 B_{ij} 左侧的最小代价, 则

$$M[1][1] = S_{01}, \quad M[1][2] = S_{02};$$

$$M[i][1] = \min \{M[i-1][1] + C_{i-1,1} + S_{i-1,1}, M[i-1][2] + C_{i-1,2} + S_{i-1,3}\}$$

$$M[i][2] = \min \{M[i-1][1] + C_{i-1,1} + S_{i-1,2}, M[i-1][2] + C_{i-1,2} + S_{i-1,4}\}$$

$$\text{甲-乙最小代价: } \min \{M[n][1] + C_{n1} + S_{n1}, M[n][2] + C_{n2} + S_{n2}\};$$

$M[j][i]$ 中存储的是甲到桥 B_{ij} 左侧的最小代价

```
Smallestcost()
```

```
{  M[1, 2, ..., n][1,2] ;
```

```
    M[1][1] = S01; M[1][2]=S02;
```

```
    for( $i=2; i \leq n; i++$ )
```

```
    { M[i][1] =min {M[i-1][1]+C $i-1,1$ +S $i-1,1$ , M[i-1][2] +C $i-1,2$ +S $i-1,3$ };
```

```
        M[i][2] =min {M[i-1][1]+C $i-1,1$ +S $i-1,2$ , M[i-1][2] +C $i-1,2$ +S $i-1,4$ };
```

```
    }
```

```
    return min {M[n][1]+C $n1$ +S $n1$ , M[n][2] +C $n2$ +S $n2$ };
```

```
}
```

Smallestcost()

```
{  T1 = S01; T2=S02;  
  for (i=2; i ≤ n; i++)  
    { T3=min{T1+Ci-1,1+Si-1,1, T2 +Ci-1,2+Si-1,3};  
      T2=min{T1+Ci-1,1+Si-1,2, T2 +Ci-1,2+Si-1,4};    T1=T3; }  
  return min{T1+Cn1+Sn1, T2 +Cn2+Sn2};  
}
```

smallestcost()

```
{ T1 = S01; T2=S02;  
  for (i=2; i ≤ n; i++)  
  { T3=T1;          T1=min{T3+Ci-1,1+Si-1,1, T2 +Ci-1,2+Si-1,3};  
    T2=min{T3+Ci-1,1+Si-1,2, T2 +Ci-1,2+Si-1,4};  
  }  
  return min{T1+Cn1+Sn1, T2 +Cn2+Sn2};  
}
```

smallestcost()

```
{ T1 = S01; T2=S02;  
  for (i=2; i ≤ n; i++)  
  { T3=min{T1+Ci-1,1+Si-1,1, T2 +Ci-1,2+Si-1,3};  
    T2=min{T1+Ci-1,1+Si-1,2, T2 +Ci-1,2+Si-1,4};    T1=T3; }  
  return min{T1+Cn1+Sn1, T2 +Cn2+Sn2};  
}
```

动态规划算法的特点

- 分解为许多重复的小问题，保存小问题的计算结果，减少重复计算
- 是递归？不是递归？

特点：

- 分析一个最优解决方案应该具备的结构
- 递归定义最优解决方案
- 自底向上构建解决算法

动态规划算法设计的步骤

- 最优解决方案的结构
- 递归的解决方案
- 自底向上的实现算法

6.2 最长公共子序列

➤ 问题

$$A = \langle A_1 A_2 \dots A_n \rangle$$

$$B = \langle B_1 B_2 \dots B_m \rangle$$

什么是子序列？什么是公共子序列？举例说明。

XJHCB, YXCCHTB

XJHCB, YXCCHTB

XJHCB, YXCCHTB

6.2 最长公共子序列

➤ 问题

$$A = \langle A_1 A_2 \dots A_n \rangle$$

$$B = \langle B_1 B_2 \dots B_m \rangle$$

蛮力的搜索方法： $\Theta(m2^n)$ 或者 $\Theta(n2^m)$

多项式时间复杂度的子序列判断算法？

6.2 最长公共子序列

- 问题: $A = \langle A_1 A_2 \dots A_n \rangle$ $B = \langle B_1 B_2 \dots B_m \rangle$
- 先求最长公共子序列的长度
- 定义 $L[i, j]$ 表示 $\langle A_1 A_2 \dots A_i \rangle$ 和 $\langle B_1 B_2 \dots B_j \rangle$ 的最长公共子序列的长度, 则
$$L[i, 0] = L[0, j] = 0;$$
问题的目标是求 $L[n, m];$

6.2 最长公共子序列

➤ 定义 $L[i, j]$ 表示 $\langle A_1 A_2 \dots A_i \rangle$ 和 $\langle B_1 B_2 \dots B_j \rangle$ 的最长公共子序列的长度, 则

$\langle A_1 A_2 \dots A_{i-1} A_i \rangle$

$\langle B_1 B_2 \dots B_{j-1} A_i \rangle$

对于任意的 $L[i, j]$, 当 $i > 0$ 且 $j > 0$, 我们有

如果 $A_i = B_j$, $L[i, j] = L[i-1, j-1] + 1$

6.2 最长公共子序列

➤ 定义 $L[i, j]$ 表示 $\langle A_1 A_2 \dots A_i \rangle$ 和 $\langle B_1 B_2 \dots B_j \rangle$ 的最长公共子序列的长度, 则

$\langle A_1 A_2 \dots A_{i-1} A_i \rangle$

$\langle B_1 B_2 \dots B_{j-1} B_j \rangle$

$\langle A_1 A_2 \dots A_{i-1} A_i \rangle$

$\langle B_1 B_2 \dots B_{j-1} \rangle$

$\langle A_1 A_2 \dots A_{i-1} \rangle$

$\langle B_1 B_2 \dots B_{j-1} B_j \rangle$

对于任意的 $L[i, j]$, 当 $i > 0$ 且 $j > 0$, 我们有

如果 $A_i = B_j$, $L[i, j] = L[i-1, j-1] + 1$

如果 $A_i \neq B_j$, $L[i, j] = \max\{L[i, j-1], L[i-1, j]\}$

6.2 最长公共子序列

➤ 问题: $A = \langle A_1 A_2 \dots A_n \rangle$ $B = \langle B_1 B_2 \dots B_m \rangle$

$$L[i, j] = \begin{cases} 0 & \text{若 } i = 0 \text{ 或 } j = 0 \\ L[i - 1, j - 1] + 1 & \text{若 } i > 0, j > 0 \text{ 和 } a_i = b_j \\ \max\{L[i, j - 1], L[i - 1, j]\} & \text{若 } i > 0, j > 0 \text{ 和 } a_i \neq b_j \end{cases}$$

[illegible]

6.2 最长公共子序列

➤ 初始化

```
1. for  $i \leftarrow 0$  to  $n$   
2.    $L[i, 0] \leftarrow 0$   
3. end for  
4. for  $j \leftarrow 0$  to  $m$   
5.    $L[0, j] \leftarrow 0$   
6. end for
```

6.2 最长公共子序列

➤ 自底向上计算

```
7. for  $i \leftarrow 1$  to  $n$ 
8.   for  $j \leftarrow 1$  to  $m$ 
9.     if  $a_i = b_j$  then  $L[i, j] \leftarrow L[i - 1, j - 1] + 1$ 
10.    else  $L[i, j] \leftarrow \max\{L[i, j - 1], L[i - 1, j]\}$ 
11.    end if
12.  end for
13. end for
14. return  $L[n, m]$ 
```

6.2 最长公共子序列

➤ 问题: $A = \langle A_1 A_2 \dots A_n \rangle$ $B = \langle B_1 B_2 \dots B_m \rangle$

➤ 如何从长度得到最长公共子序列?

➤ 时间复杂度

➤ 空间复杂度: 能否只使用一维数组?

6.2 最长公共子序列

➤ 如何从长度得到最长公共子序列?

$$L[i, j] = \begin{cases} 0 & \text{若 } i = 0 \text{ 或 } j = 0 \\ L[i - 1, j - 1] + 1 & \text{若 } i > 0, j > 0 \text{ 和 } a_i = b_j \\ \max\{L[i, j - 1], L[i - 1, j]\} & \text{若 } i > 0, j > 0 \text{ 和 } a_i \neq b_j \end{cases}$$

		<i>j</i>									
			X	X	Y	T	Y	P	Q	Y	Y
<i>i</i>		0	1	2	3	4	5	6	7	8	9
0		0	0	0	0	0	0	0	0	0	0
X	1	0	1	1	1	1	1	1	1	1	1
Y	2	0	1	1	2	2	2	2	2	2	2
Q	3	0	1	1	2	2	2	2	3	3	3
P	4	0	1	1	2	2	2	3	3	3	3
Y	5	0	1	1	2	2	3	3	3	4	4

6.2 最长公共子序列

➤ 空间复杂度：能否只使用一维数组？

$$L[i, j] = \begin{cases} 0 & \text{若 } i = 0 \text{ 或 } j = 0 \\ L[i - 1, j - 1] + 1 & \text{若 } i > 0, j > 0 \text{ 和 } a_i = b_j \\ \max\{L[i, j - 1], L[i - 1, j]\} & \text{若 } i > 0, j > 0 \text{ 和 } a_i \neq b_j \end{cases}$$

[illegible]

```

LCS( )
{  Q[1,2,..., n] = 0;
   for(j=1; j ≤ m; j++)
   {   if(Bj=A1)  T=1;
       else  T = Q[1];
       for(i=2; i ≤ n; i++)
       {   if(Bj=Ai) { S = Q[i-1]+1; Q[i-1] = T ; T = S;}
           else { Q[i-1] = T ; T = max {T, Q[i]}}; }
       }
       Q[n] = T;
   }
   return Q[n];
}

```

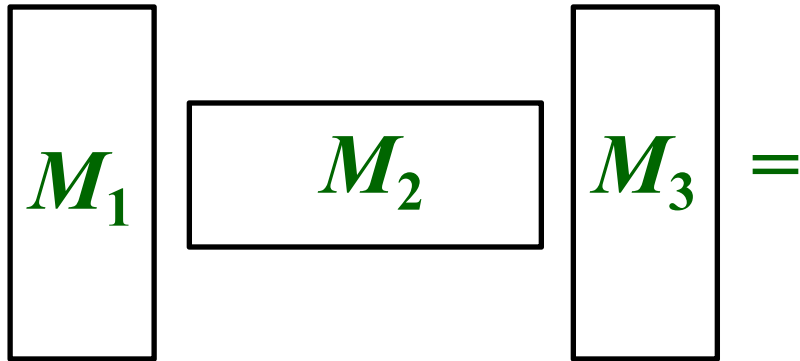
6.3 矩阵链相乘

- 问题： 求出计算 n 个矩阵相乘 $M_1M_2\dots M_n$ 所需的最少的乘法次数
- 如果 M_1M_2 分别是 $a \times b$ 和 $b \times c$ 的，那么计算 M_1M_2 分需要的数量乘法为 $a \times b \times c$ 次

6.3 矩阵链相乘

➤ 一个小例子： 计算 $M_1M_2M_3$

其维度分别是： $10 \times 2, 2 \times 10, 10 \times 2$



➤ 第一种计算顺序： $M_1M_2M_3=(M_1M_2)M_3$

需要的乘法的次数为： $200+200=400$

➤ 第二种计算顺序： $M_1M_2M_3=M_1(M_2M_3)$

需要的乘法的次数为： $40+40=80$

6.3 矩阵链相乘

➤ 蛮力法：列举每一种可能的结合顺序，比较其所需的乘法次数

$$(M_1(M_2(M_3M_4)))$$

$$(M_1((M_2M_3)M_4))$$

$$((M_1M_2)(M_3M_4))$$

$$(((M_1M_2)M_3)M_4)$$

$$((M_1(M_2M_3))M_4)$$

假设 n 个矩阵相乘可能的结合顺序有 $f(n)$ 种，则

$$\underbrace{(M_1M_2\cdots M_k)}_{f(k)} \times \underbrace{(M_{k+1}M_{k+2}\cdots M_n)}_{f(n-k)}$$

$f(k)*f(n-k)$, 其中 $k=1, 2, \dots, n-1$

$$f(n) = \sum_{k=1}^{n-1} f(k)f(n-k)$$

$$f(2) = 1, f(3) = 2$$

6.3 矩阵链相乘

➤ 蛮力法：列举每一种可能的结合顺序，比较其所需的乘法次数

假设 n 个矩阵相乘可能的结合顺序有 $f(n)$ 种，则

$$f(n) = \sum_{k=1}^{n-1} f(k)f(n-k)$$

$$f(n) = \Omega(4^n/n^{1.5})$$

$$f(2) = 1, f(3) = 2$$

$$f(n) = \frac{1}{n} \binom{2n-2}{n-1}$$

Catalan数 $C_n = f(n+1)$

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16 796, ...

6.3 矩阵链相乘

- 问题：求出计算 n 个矩阵相乘 $M_1M_2\dots M_n$ 所需的最少的乘法次数
- 有没有多项式时间的算法，找出矩阵链乘所需要的最少数量的乘法次数？

6.3 矩阵链相乘

➤ 问题：求出计算 n 个矩阵相乘 $M_1M_2\dots M_n$ 所需的最少的乘法次数

➤ 动态规划：

数组 $r[1, 2, \dots, n+1]$ 中存储每个矩阵的行数和列数

设置 $r[i]$ 存储矩阵 M_i 的行数，则

$i > 1$ 时， $r[i]$ 必然也是矩阵 M_{i-1} 的列数

设置 $r[n+1]$ 存储矩阵 M_n 的列数

6.3 矩阵链相乘

➤ 问题: $M_1M_2\cdots M_n$

➤ 动态规划: 最优解的结构;

记 $M_{i,j}=M_iM_{i+1}\cdots M_j$,

$C[i,j]$ 是计算 $M_{i,j}$ 所需要的数量乘法的最少次数

$$[M_iM_{i+1}\cdots M_{k-1}][M_kM_{k+1}\cdots M_j]$$

$$C[i, k-1]$$

$$C[k, j]$$

其中 $k=i+1, \dots, j$

$$r[i]*r[k]*r[j+1]$$

$$C[i, j]=\min_{i < k \leq j} \{C[i, k-1] + C[k, j] + r[i]*r[k]*r[j+1]\}$$

6.3 矩阵链相乘

➤ 问题: $M_1M_2\cdots M_n$

➤ 动态规划: 最优解的结构;

记 $M_{i,j}=M_iM_{i+1}\cdots M_j$,

$C[i,j]$ 表示计算所需要的数量乘法的最少次数

$$C[i,j]=\min_{i < k \leq j} \{C[i,k-1] + C[k,j] + r[i] * r[k] * r[j+1]\}$$

➤ 这里要求出的是 $C[1, n]$

且 $C[i, i] = 0$

6.3 矩阵链相乘

➤ 问题: $M_1M_2\dots M_n$

➤ 动态规划: 自底向上的算法求 $C[1, n]$

按照什么顺序呢?

先回头看看计算

$C[i, j]$ 需要什么

$C[1,1]$	$C[1,2]$	$C[1,3]$	$C[1,4]$	$C[1,5]$	$C[1,6]$
	$C[2,2]$	$C[2,3]$	$C[2,4]$	$C[2,5]$	$C[2,6]$
		$C[3,3]$	$C[3,4]$	$C[3,5]$	$C[3,6]$
			$C[4,4]$	$C[4,5]$	$C[4,6]$
				$C[5,5]$	$C[5,6]$
					$C[6,6]$

➤ 横? 竖? 斜?

$$C[i, j] = \min_{i < k \leq j} \{C[i, k-1] + C[k, j] + r[i] * r[k] * r[j+1]\}$$

```

MatrixChain( )
{
    for( $i=1; i \leq n; i++$ )  $C[i,i] = 0$ ;
    for( $d=1; d \leq n-1; d++$ )
        for(  $i=1; i \leq n-d; i++$ )
            {
                 $j = i + d$ ;
                 $C[i,j] = C[i+1,j] + r[i] * r[i+1] * r[j+1]$ ;
                for(  $k=i+2; k \leq j; k++$ )
                     $C[i,j] = \min \{ C[i,j],$ 
                         $C[i, k-1] + C[k, j] + r[i] * r[k] * r[j+1] \}$ ;
            }
    return  $C[1, n]$ ;
}

```

时间复杂度? $O(n^3)$

6.3 矩阵链相乘

➤ 例子:

$M_1: 5 \times 10, M_2: 10 \times 4, M_3: 4 \times 6, M_4: 6 \times 10, M_5: 10 \times 2$

$C[1,1]=$	$C[1,2]=$	$C[1,3]=$	$C[1,4]=$	$C[1,5]=$
	$C[2,2]=$	$C[2,3]=$	$C[2,4]=$	$C[2,5]=$
		$C[3,3]=$	$C[3,4]=$	$C[3,5]=$
			$C[4,4]=$	$C[4,5]=$
				$C[5,5]=$

6.5 所有点的最短路径

➤ 问题: $G=(V, E)$ 有向带权图

$l[i, j]$ 为 i 到 j 的边的权值, 如果没有边则为 $+\infty$

➤ V 包含了 n 个顶点, 记为 $\{1, 2, \dots, n\}$

➤ 求出任意两个顶点之间的最短路径长度

➤ 最终计算结果的存储: $n*n$ 的二维数组,

比如 $D[1, 2, \dots, n][1, 2, \dots, n]$, 其中 $D[i][j]$ 表示 i 到 j 的最短路径长度

$$D_0[i][j] = l[i, j],$$

$$D_n[1, 2, \dots, n][1, 2, \dots, n]$$

6.5 所有点的最短路径

➤ 问题: $G=(V, E)$ 有向带权图

$l[i, j]$ 为 i 到 j 的边的权值, 如果没有边则为 $+\infty$

➤ V 包含了 n 个顶点, 记为 $\{1, 2, \dots, n\}$

➤ 求出任意两个顶点之间的最短路径长度

➤ 最终计算结果的存储: $n*n$ 的二维数组,

比如 $D[1, 2, \dots, n][1, 2, \dots, n]$, 其中 $D[i][j]$ 表示 i 到 j 的最短路径长度

6.5 所有点的最短路径

➤ 问题： $G=(V, E)$ 有向带权图

$l[i, j]$ 为 i 到 j 的边的权值，如果没有边则为 $+\infty$

➤ Floyd 算法：

$D_k[1, 2, \dots, n][1, 2, \dots, n]$ 的含义

$$D_k[i][j] = d_{i,j}^k$$

$d_{i,j}^k$ 是从 i 到 j ，并且不经过 $\{k+1, k+2, \dots, n\}$ 中任何顶点的最短路径的长度

6.5 所有点的最短路径

➤ 问题： $G=(V, E)$ 有向带权图

$l[i, j]$ 为 i 到 j 的边的权值，如果没有边则为 $+\infty$

➤ Floyd 算法：

$$D_0[i][j] = l[i, j], \quad d_{i,j}^0 = l[i, j]$$

$$d_{i,j}^1$$

$$d_{i,j}^2$$

$D_n[1, 2, \dots, n][1, 2, \dots, n]$ 就是要求的最终结果

6.5 所有点的最短路径

➤ 问题： $G=(V, E)$ 有向带权图

$l[i, j]$ 为 i 到 j 的边的权值，如果没有边则为 $+\infty$

➤ Floyd算法： 考虑 $d_{i,j}^k$ ， $1 \leq k \leq n$

$d_{i,j}^k$ 与 $d_{i,j}^{k-1}$ 的区别在于？是否允许经过顶点 k

➤ 计算 $d_{i,j}^k$ 有两种可能：(1) 不经过 k , (2) 经过 k

$$d_{i,j}^k = \min \{ d_{i,j}^{k-1}, d_{i,k}^{k-1} + d_{k,j}^{k-1} \}, \text{ 若 } 1 \leq k \leq n$$

$$D_{k-1} \rightarrow D_k$$

6.5 所有点的最短路径

➤ 问题： $G=(V, E)$ 有向带权图

$l[i, j]$ 为 i 到 j 的边的权值，如果没有边则为 $+\infty$

➤ Floyd算法： 考虑 $d_{i,j}^k$ ， $1 \leq k \leq n$

➤ 从 i 到 j 有两种可能：(1) 不经过 k , (2) 经过 k

$$d_{i,j}^k = \min \{ d_{i,j}^{k-1}, d_{i,k}^{k-1} + d_{k,j}^{k-1} \}, \text{ 若 } 1 \leq k \leq n$$

$$D_{k-1} \rightarrow D_k$$

$$D_0 \rightarrow D_1 \rightarrow D_2 \rightarrow \dots \rightarrow D_n$$

6.5 所有点的最短路径

➤ 问题: $G=(V, E)$ 有向带权图, $l[i, j]$ 为 i 到 j 的边权值

➤ Floyd 算法: $d_{i,j}^0 = l[i, j]$

$$d_{i,j}^k = \min \{d_{i,j}^{k-1}, d_{i,k}^{k-1} + d_{k,j}^{k-1}\}, \text{ 若 } 1 \leq k \leq n$$

$$D_0 \rightarrow D_1 \rightarrow D_2 \rightarrow \dots \rightarrow D_n$$

用权值 $l[i, j]$ 初始化数组 D_0 ;

for($k=1; k \leq n; k++$)

for($i=1; i \leq n; i++$)

for($j=1; j \leq n; j++$)

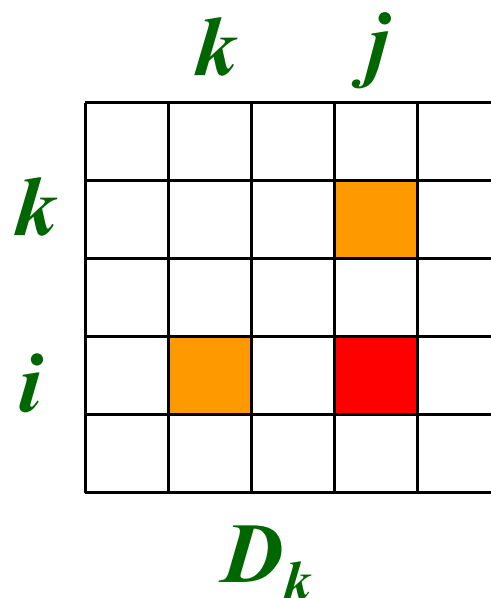
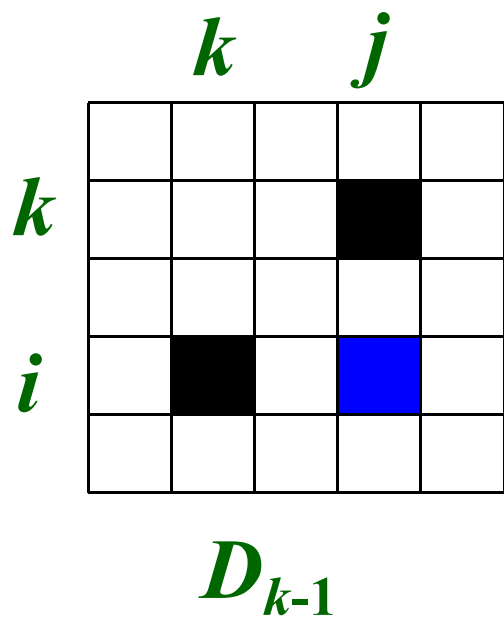
$$D_k[i][j] = \min \{D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j]\};$$

◆ 能否减少空间开销?
只使用一个数组

$$d_{i,k}^k = \min \{ d_{i,k}^{k-1}, \quad d_{i,k}^{k-1} + d_{k,k}^{k-1} \} = d_{i,k}^{k-1}$$

$$d_{k,j}^k = \min \{ d_{k,j}^{k-1}, \quad d_{k,k}^{k-1} + d_{k,j}^{k-1} \} = d_{k,j}^{k-1}$$

$$d_{i,j}^k = \min \{ d_{i,j}^{k-1}, \quad d_{i,k}^{k-1} + d_{k,j}^{k-1} \}, \quad \text{若 } 1 \leq k \leq n$$



6.5 所有点的最短路径

➤ 问题: $G=(V, E)$ 有向带权图, $l[i, j]$ 为 i 到 j 的边权值

➤ Floyd 算法: $d_{i,j}^0 = l[i, j]$

$$d_{i,j}^k = \min \{d_{i,j}^{k-1}, d_{i,k}^{k-1} + d_{k,j}^{k-1}\}, \text{ 若 } 1 \leq k \leq n$$

用权值 $l[i, j]$ 初始化数组 D ;

// 只使用一个数组 D

```
for(k=1; k ≤ n; k++)
```

```
    for(i=1; i ≤ n; i++)
```

```
        for(j=1; j ≤ n; j++)
```

```
             $D[i][j] = \min \{D[i][j], D[i][k] + D[k][j]\};$ 
```

例 7.5 考虑图 7.4 所示的有向图

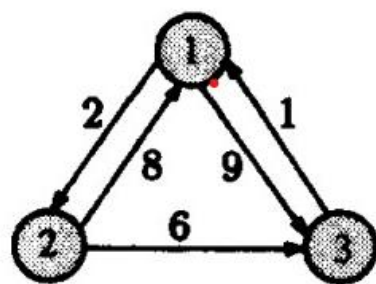


图 7.4 所有点对最短路径问题的实例

矩阵 D_0, D_1, D_2 和 D_3 是

$$D_0 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & \infty & 0 \end{bmatrix} \quad D_1 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

$$D_2 = \begin{bmatrix} 0 & 2 & 8 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix} \quad D_3 = \begin{bmatrix} 0 & 2 & 8 \\ 7 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

6.6 背包问题

➤ 问题：物品集 $U=\{u_1, u_2, \dots, u_n\}$ ，背包容量为 C
 s_i 和 v_i 分别为 u_i 的体积和价值，求出 U 的子集 S ，
使得

$$\sum_{u_i \in S} v_i \quad \text{最大}$$

$$\text{且} \quad \sum_{u_i \in S} s_i \leq C$$

常用于密码算法设计中。

6.6 背包问题

- 问题：物品集 $U=\{u_1, u_2, \dots, u_n\}$ ，背包容量为 C
 s_i 和 v_i 分别为 u_i 的体积和价值，求出 U 的子集 S
- 定义 $V[i, j]$ 表示从前 i 个物品 $\{u_1, u_2, \dots, u_i\}$ ，背包体积为 j 时，能够获得最大价值。

那么，若 $i = 0$ 或 $j = 0$ ， $V[i, j] = 0$,

目标是计算 $V[n, C]$

6.6 背包问题

- 问题：物品集 $U=\{u_1, u_2, \dots, u_n\}$ ，背包容量为 C
 s_i 和 v_i 分别为 u_i 的体积和价值，求出 U 的子集 S
- 定义 $V[i, j]$ 表示从前 i 个物品 $\{u_1, u_2, \dots, u_i\}$ ，背包体积为 j 时，能够获得最大价值。

那么，若 $i = 0$ 或 $j = 0$ ， $V[i, j] = 0$,

若 $j < s_i$ ， $V[i, j] = V[i-1, j]$

若 $i > 0$ 且 $j \geq s_i$ ，有两种可能：(1) 不装 u_i , (2) 装 u_i

$$V[i, j] = \max \{V[i-1, j], V[i-1, j-s_i] + v_i\},$$


6.6 背包问题

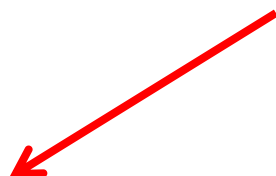
➤ 问题：物品集 $U = \{u_1, u_2, \dots, u_n\}$ ，背包容量为 C
 s_i 和 v_i 分别为 u_i 的体积和价值，求出 U 的子集 S

$$V[i, j] = \begin{cases} 0, & \text{若 } i = 0 \text{ 或 } j = 0 \\ V[i-1, j], & \text{若 } j < s_i \\ \max \{V[i-1, j], V[i-1, j-s_i] + v_i\}, & \text{若 } i > 0 \text{ 且 } j \geq s_i \end{cases}$$

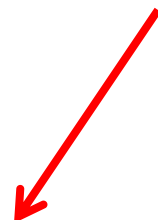
[illegible]

```
1. for  $i \leftarrow 0$  to  $n$ 
2.    $V[i, 0] \leftarrow 0$ 
3. end for
4. for  $j \leftarrow 0$  to  $C$ 
5.    $V[0, j] \leftarrow 0$ 
6. end for
```

初始化



自底向上计算



```
7. for  $i \leftarrow 1$  to  $n$ 
8.   for  $j \leftarrow 1$  to  $C$ 
9.      $V[i, j] \leftarrow V[i - 1, j]$ 
10.    if  $s_i \leq j$  then  $V[i, j] \leftarrow \max\{V[i, j], V[i - 1, j - s_i] + v_i\}$ 
11.   end for
12. end for
13. return  $V[n, C]$ 
```

6.6 背包问题

➤ 问题：物品集 $U=\{u_1, u_2, \dots, u_n\}$ ，背包容量为 C
 s_i 和 v_i 分别为 u_i 的体积和价值，求出 U 的子集 S

时间复杂度： $O(n * C)$

伪多项式的

能只使用一维数组吗？

6.6 背包问题

➤ 问题：物品集 $U = \{u_1, u_2, \dots, u_n\}$ ，背包容量为 C
 s_i 和 v_i 分别为 u_i 的体积和价值，求出 U 的子集 S

$$V[i, j] = \begin{cases} 0, & \text{若 } i = 0 \text{ 或 } j = 0 \\ V[i-1, j], & \text{若 } j < s_i \\ \max \{V[i-1, j], V[i-1, j-s_i] + v_i\}, & \text{若 } i > 0 \text{ 且 } j \geq s_i \end{cases}$$

[illegible]

KNAPSACK()

```
{  W[0,1,2,..., C] = 0;
    Q[0,1,2,..., C] = 0;
  for( $i=1$ ;  $i \leq n$ ;  $i++$ )
  {   for( $j=1$ ;  $j \leq C$ ;  $j++$ )
      {   Q[j] = W[j];
          if( $s_i \leq j$ )  Q[j] = max{Q[j], W[j- $s_i$ ]+ $v_i$ };
      }
      W[1,2,..., C] = Q[1,2,..., C];
  }
  return W[C];
}
```

还能进一步减少空间
开销吗?

KNAPSACK()

```
{  W[0,1,2,..., C] = 0;
    for(i = 1; i ≤ n; i++)
    {    for(j = C; j ≥ 1; j--){
            if(si ≤ j)  W[j] = max{W[j], W[j-si]+vi};
        }
    }
    return W[C];
}
```

四、某学生甲平时不认真学习，打算在期末考试前进行突击复习。假设学生甲共有 n 门课要考试，每门课的学分分别是 $X_1, X_2, \dots, X_n$ ，如果想各门课程的考试及格，需要的复习时间分别是 $T_1, T_2, \dots, T_n$ 小时，即学生甲如果想第 i 门课程考试及格，需要花费 T_i 个小时复习该门课程。然而，学生甲共计只有 C 小时的时间用来复习所有的课程。请使用动态规划的思想，设计一个 $O(n \cdot C)$ 的算法，求学生甲所能考及格的课程学分之和的最大值。（提示：类似背包问题）要求：

- (1) 给出上述问题最优解的表示方法和递推关系式；
- (2) 给出算法的伪代码，并添加必要的注释；
- (3) 分析所设计算法的时间复杂度。

以上是作业，4月30号课前交